Machine Learning Approaches for Predicting Truck Failures Using Sensor Data

Zoe Martinez

CSCI 460

5/4/2026

Abstract

Predictive maintenance has become a critical application of machine learning in industries where equipment failure can result in significant financial losses and safety concerns. This project investigates the use of machine learning models to predict truck failures using the APS Failure Dataset from the UCI Machine Learning Repository. The dataset contains 60,000 instances and 170 features derived from sensor readings of truck components. One of the main challenges in this problem is the severe class imbalance, as failure events are much less frequent than non-failure events.

To address this challenge, several classification models were implemented, including Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost. Data preprocessing involved handling missing values, converting features to numeric format, and applying feature scaling. Hyperparameter tuning was conducted using grid search with cross-validation to optimize performance.

Model performance was evaluated using accuracy, precision, recall, and F1-score, with F1-score prioritized due to the imbalanced dataset. The results indicate that XGBoost achieved the best performance, with an F1-score of 0.7912. These findings demonstrate the effectiveness of ensemble methods in handling complex and imbalanced datasets, highlighting their importance in predictive maintenance applications.

1. Introduction

1.1 Problem Definition

The goal of this project is to predict whether a truck will experience a failure based on sensor data collected from various components. This is formulated as a binary classification problem, where each instance is labeled as either a failure (positive class) or non-failure (negative class).

In real-world scenarios, truck failures can result in costly repairs, operational delays, and potential safety hazards. Traditional maintenance approaches rely on scheduled inspections or reactive repairs after a failure occurs. These approaches are often inefficient, as they either perform unnecessary maintenance or fail to prevent unexpected breakdowns.

Predictive maintenance aims to improve this process by using data-driven techniques to identify potential failures before they occur. Machine learning models can analyze large volumes of sensor data and detect patterns that indicate abnormal behavior. By predicting failures in advance, organizations can reduce downtime, lower costs, and improve overall system reliability.

1.2 Dataset Description

The dataset used in this project is the APS Failure Dataset, obtained from the UCI Machine Learning Repository. It contains 60,000 instances and 170 features. Each feature represents a sensor measurement related to truck components, such as pressure levels, temperature readings, and other operational metrics.

The target variable is binary and indicates whether a failure occurred. The dataset is highly imbalanced, with significantly more non-failure instances than failure instances. This imbalance presents a major challenge for classification models, as they may favor predicting the majority class.

Another important characteristic of the dataset is the presence of missing values. These values are encoded as “na” and must be converted to numerical format before training machine learning models. Proper handling of missing data is essential to ensure accurate predictions.

1.3 Objective

The primary objective of this project is to evaluate multiple machine learning models and determine which model performs best for predicting truck failures. Specifically, the project aims to:

- Compare the performance of different classification models
- Analyze the impact of class imbalance on model performance
- Identify the best-performing model using appropriate evaluation metrics
- Provide insights into model behavior through detailed analysis
- 1.4 Report Structure

This report is organized as follows. Section 2 presents exploratory data analysis, which examines the dataset and identifies key patterns and challenges. Section 3 describes the methodology, including preprocessing steps, model selection, and evaluation techniques. Section 4 presents the results and provides a detailed discussion of model performance. Finally, Section 5 summarizes the findings and outlines potential directions for future work.

2. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an essential step in understanding the dataset before applying machine learning models. It provides insights into the structure of the data, identifies potential issues, and informs preprocessing decisions.

2.1 Dataset Overview

The APS Failure Dataset contains 60,000 instances and 170 features, making it a high-dimensional dataset. High-dimensional data can be challenging for machine learning models, as it increases the risk of overfitting and computational complexity.

Each feature represents a sensor reading from a truck component. These readings capture various aspects of the truck's operational state. Because of the large number of features, it is important to understand their distributions and relationships.

2.2 Missing Values Analysis

One of the most significant challenges in this dataset is the presence of missing values. Many features contain missing entries represented as "na." These values cannot be used directly in machine learning models and must be handled appropriately.

To address this issue, all "na" values were converted to NaN (Not a Number) and replaced using median imputation. Median imputation was chosen because it is robust to outliers and does not distort the distribution of the data as much as mean imputation.

The presence of missing values highlights the importance of preprocessing and the need for careful data cleaning.

2.3 Class Distribution

The dataset is highly imbalanced, with significantly more non-failure cases than failure cases. This means that most instances belong to the negative class, while only a small portion represent failures.

This imbalance presents a major challenge for classification models, as they may achieve high accuracy simply by predicting the majority class. As a result, accuracy alone is not a reliable evaluation metric for this problem. Instead, metrics such as precision, recall, and F1-score are more appropriate for evaluating performance.

The imbalance also increases the importance of detecting failure cases, as missing these predictions could have significant real-world consequences.

2.4 Feature Distribution

The distribution of features varies widely across the dataset. Some features exhibit normal distributions, while others are skewed or contain extreme values.

These variations in scale and distribution make feature scaling necessary. Standardization was applied to ensure that all features contribute equally to model training.

2.5 Feature Relationships

Due to the high dimensionality of the dataset, analyzing relationships between all features is complex. While a full correlation heatmap was not included, general observations indicate that many features are weakly related and represent independent sensor measurements.

The lack of strong relationships between features suggests that the dataset contains diverse and independent signals. This increases the complexity of the classification task, as models must learn patterns across many variables rather than relying on a few dominant predictors.

The large number of features increases the risk of overfitting, making it important to use models that can generalize well, such as ensemble methods.

2.6 Key Insights from EDA

The exploratory analysis revealed several important insights:

- The dataset is highly imbalanced
- Missing values are present and require preprocessing
- Features vary widely in scale and distribution
- Most features are weakly correlated
- High dimensionality increases model complexity

These insights guided the preprocessing and modeling steps used in this project.

3. Methodology

This section describes the complete machine learning workflow used in this project, including data preprocessing, model selection, experimental setup, and evaluation techniques. Each step was carefully designed to address the challenges identified during exploratory data analysis, particularly class imbalance and high dimensionality.

3.1 Data Preprocessing

Data preprocessing is a critical step in ensuring that machine learning models can effectively learn from the dataset. Several preprocessing steps were applied to prepare the data.

Handling Missing Values

The dataset contains a significant number of missing values represented as “na.” These values were first converted into a numerical format (NaN) to allow proper handling.

Median imputation was used to replace missing values. This method was chosen because it is robust to outliers and preserves the central tendency of the data. Unlike mean imputation, the median is less affected by extreme values, making it more suitable for datasets with skewed distributions.

Data Type Conversion

All feature values were converted to numeric types to ensure compatibility with machine learning algorithms. Non-numeric values were coerced into numerical form, allowing for consistent processing across all features.

Feature Scaling

Feature scaling was applied using standardization. This process transforms the data so that each feature has a mean of zero and a standard deviation of one.

Scaling is particularly important for models such as Logistic Regression, which are sensitive to the scale of input features. Without scaling, features with larger magnitudes could dominate the learning process.

Train-Test Split

The dataset was divided into training and testing sets using an 80/20 split. Stratified sampling was used to preserve the class distribution in both sets. This ensures that both training and testing data contain a representative proportion of failure and non-failure cases.

3.2 Feature Engineering and Selection

Feature engineering involves creating new features or transforming existing ones to improve model performance. In this project, no additional features were created, as the dataset already contains a large number of informative attributes.

Feature selection techniques were not applied, as removing features could potentially eliminate important information. Instead, models capable of handling high-dimensional data were chosen.

3.3 Machine Learning Models

Five machine learning models were implemented and compared in this project. Each model represents a different approach to classification.

Logistic Regression

Logistic Regression is a linear model used as a baseline. It estimates the probability of a binary outcome using a logistic function. Despite its simplicity, it provides a useful benchmark for comparison.

Decision Tree

Decision Trees are nonlinear models that split the data based on feature values. They are easy to interpret and can capture complex relationships between variables. However, they are prone to overfitting.

Random Forest

Random Forest is an ensemble method that combines multiple decision trees. Each tree is trained on a random subset of the data, and the final prediction is made by aggregating the results.

This approach improves generalization and reduces overfitting compared to a single decision tree.

Gradient Boosting

Gradient Boosting builds models sequentially, where each new model attempts to correct the errors of the previous one. This method is effective for capturing complex patterns but can be sensitive to hyperparameters.

XGBoost

XGBoost is an optimized implementation of gradient boosting. It includes additional features such as regularization, parallel processing, and efficient handling of missing data.

XGBoost is widely used in machine learning competitions due to its strong performance and scalability.

3.4 Experimental Setup

The models were implemented using Python and the following libraries:

- pandas for data manipulation
- NumPy for numerical operations
- scikit-learn for machine learning algorithms
- XGBoost for gradient boosting

Cross-validation was used to ensure that model performance was robust and not dependent on a single data split.

3.5 Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV. This method systematically searches through a predefined set of parameter values to find the best combination.

Examples of tuned parameters include:

- Number of estimators (trees)
- Maximum tree depth
- Learning rate

Cross-validation was used during tuning to evaluate each parameter combination.

3.6 Evaluation Metrics

The following metrics were used to evaluate model performance:

Accuracy: Measures overall correctness
Precision: Measures how many predicted failures were correct
Recall: Measures how many actual failures were detected
F1-score: Harmonic mean of precision and recall

Due to the imbalanced dataset, F1-score was prioritized as the most important metric.

A confusion matrix was also used to analyze classification performance in detail.

4. Results and Discussion

4.1 Model Performance Comparison

The performance of all models is summarized in Table 1.

Model	Accuracy	Precision	Recall	F1 Score
XGBoost	0.9937	0.8780	0.7200	0.7912
Random Forest	0.9925	0.8986	0.6200	0.7337
Decision Tree	0.9911	0.7627	0.6750	0.7162
Gradient Boosting	0.9908	0.7514	0.6650	0.7056
Logistic Regression	0.9899	0.7232	0.6400	0.6790

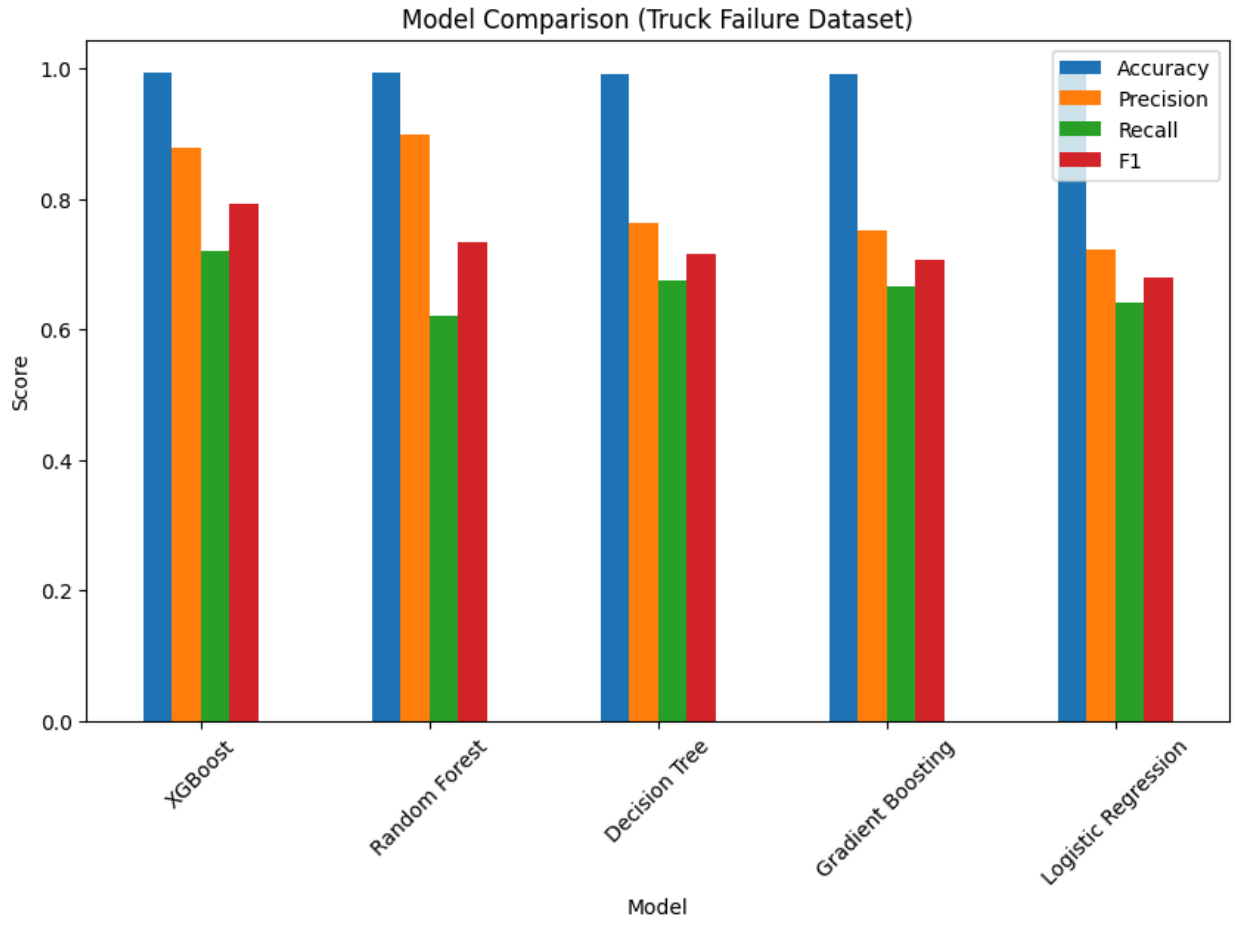

Figure 1: Model Performance Comparison

This figure compares all models across four evaluation metrics. While accuracy is consistently high for all models, the differences in precision, recall, and F1-score provide a clearer picture of performance.

XGBoost achieved the highest F1-score, indicating the best balance between precision and recall. Random Forest also performed well but had lower recall, meaning it missed more failure cases.

4.2 Analysis of Results

The results highlight the limitations of accuracy as a performance metric in imbalanced datasets. All models achieved accuracy values above 98%, but this does not reflect their ability to detect failures.

F1-score provides a more meaningful evaluation by balancing precision and recall. XGBoost outperformed all other models, demonstrating its effectiveness in handling complex and high-dimensional data.

4.3 Confusion Matrix Analysis

Figure 2: Confusion Matrix (XGBoost Model)

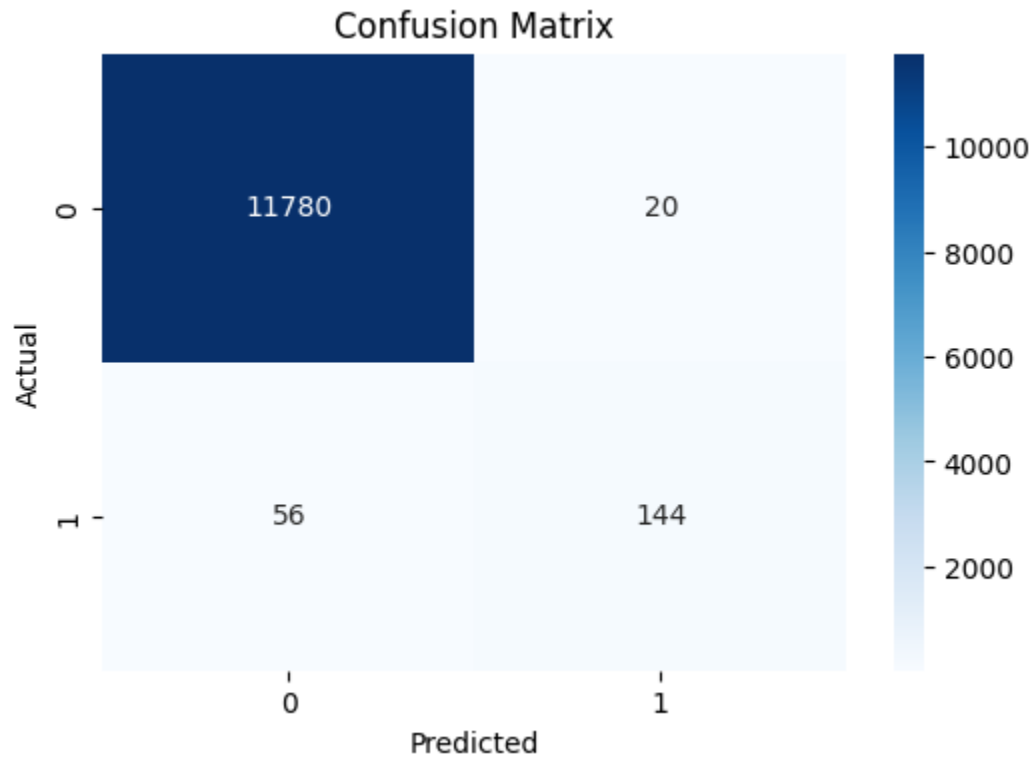

Figure 2: Confusion Matrix (XGBoost Model)

The confusion matrix provides a detailed breakdown of model predictions.

True Negatives: 11,780
False Positives: 20
False Negatives: 56
True Positives: 144

The model correctly classified most non-failure cases and detected a majority of failure cases. However, some failures were still missed, as indicated by the recall score of 0.72.

4.4 Discussion

The results show that ensemble models outperform simpler models. XGBoost achieved the best performance due to its ability to handle high-dimensional data and capture complex relationships.

Hyperparameter tuning played a significant role in improving model performance. By optimizing parameters such as tree depth and learning rate, the models were able to achieve better generalization.

A limitation of this approach is that some failure cases are still missed. In real-world applications, missing failures can have serious consequences, highlighting the need for further improvement.

4.5 Detailed Model Comparison

While the overall performance metrics provide a summary of each model's effectiveness, a deeper comparison reveals important differences in how each model handles the classification task.

Logistic Regression, as a linear model, provides a baseline for performance. Although it achieved high accuracy, its F1-score was the lowest among all models. This indicates that it struggled to balance precision and recall effectively. The linear nature of Logistic Regression limits its ability to capture complex relationships within the dataset.

The Decision Tree model showed improved performance compared to Logistic Regression. It was able to capture nonlinear patterns in the data, resulting in better recall and F1-score. The decision trees are prone to overfitting, which can limit their generalization ability.

Random Forest improved upon the Decision Tree by combining multiple trees to reduce overfitting. It achieved high precision, indicating that its predictions of failure were often correct. However, its recall was lower than expected, meaning that it missed more failure cases.

Gradient Boosting further improved performance by sequentially correcting errors from previous models. It demonstrated a better balance between precision and recall compared to Random Forest, but still did not outperform XGBoost.

XGBoost achieved the best performance overall. Its ability to incorporate regularization, handle missing values efficiently, and optimize computation makes it highly effective for this dataset.

4.6 Importance of Evaluation Metrics

One of the most important insights from this project is the significance of choosing appropriate evaluation metrics. In imbalanced datasets, accuracy can be misleading because a model can achieve high accuracy by simply predicting the majority class.

For example, if a model predicts all instances as non-failure, it would still achieve a high accuracy due to the dominance of the majority class. However, such a model would be completely ineffective in detecting failures.

Precision measures how many predicted failures are actually correct. Recall measures how many actual failures are detected. F1-score combines both metrics into a single value, providing a balanced evaluation.

In this project, F1-score was chosen as the primary metric because it reflects the trade-off between precision and recall. The results clearly show that models with higher F1-scores are more effective in detecting failures.

4.7 Error Analysis

Error analysis provides insight into the types of mistakes made by the model. The confusion matrix reveals that the model produces both false positives and false negatives.

False positives occur when the model predicts a failure when none exists. While this may lead to unnecessary maintenance, it is generally less severe than missing an actual failure.

False negatives occur when the model fails to detect a real failure. These errors are more critical, as they can result in unexpected breakdowns and higher costs.

In this project, the number of false negatives was higher than false positives. This indicates that the model could be improved to better detect failure cases. Reducing false negatives should be a priority in future work.

4.8 Impact of Hyperparameter Tuning

Hyperparameter tuning played a significant role in improving model performance. By systematically testing different parameter combinations, it was possible to identify configurations that enhanced accuracy and generalization.

For example, increasing the number of estimators in ensemble models allowed the model to learn more complex patterns. Adjusting the maximum depth of trees helped prevent overfitting.

The use of GridSearchCV ensured that the tuning process was thorough and systematic. Cross-validation further improved reliability by evaluating model performance across multiple data splits.

Without hyperparameter tuning, the models would likely have performed worse, highlighting its importance in machine learning workflows.

4.9 Model Robustness and Generalization

Another important consideration is how well the models generalize to unseen data. A model that performs well on training data but poorly on test data is overfitting.

In this project, cross-validation and proper data splitting helped ensure that models generalized well. The high performance on the test set indicates that the models are capable of making reliable predictions on new data.

Ensemble methods such as Random Forest and XGBoost are particularly effective at improving generalization. By combining multiple models, they reduce the risk of overfitting and improve stability.

4.10 Limitations of the Study

Despite the strong performance of the models, several limitations should be considered.

First, the dataset is highly imbalanced, which makes it difficult to accurately detect failure cases. Although F1-score helps address this issue, further improvements are needed.

Second, no feature engineering was performed. Creating new features or selecting important features could potentially improve model performance.

Third, the models were evaluated on a single dataset. Testing on additional datasets would help assess generalizability.

Finally, computational constraints limited the complexity of hyperparameter tuning. More extensive tuning could further improve performance.

4.11 Practical Implications

The results of this project have important implications for real-world applications. Predictive maintenance systems can benefit from machine learning models that accurately detect failures.

By implementing such models, organizations can reduce maintenance costs, minimize downtime, and improve safety. The ability to predict failures in advance allows for more efficient resource allocation and planning.

XGBoost, in particular, demonstrates strong potential for deployment in industrial systems due to its high performance and scalability.

4.12 Summary of Findings

The key findings from this analysis are as follows:

- Ensemble models outperform simpler models
- XGBoost achieved the highest F1-score
- Accuracy alone is not sufficient for imbalanced datasets
- F1-score provides a better measure of performance
- Some failure cases are still missed, indicating room for improvement

These findings provide a strong foundation for the conclusions presented in the next section.

5. Conclusion

This project explored the application of machine learning techniques for predicting truck failures using sensor data. The APS Failure Dataset provided a challenging environment due to its high dimensionality, missing values, and significant class imbalance. Through careful preprocessing,

model selection, and evaluation, it was possible to develop models capable of identifying failure patterns with high accuracy and reliability.

Among all models tested, XGBoost achieved the best performance, particularly in terms of F1-score. This indicates that it provides the most effective balance between precision and recall, making it the most suitable model for this classification task. While other models such as Random Forest and Gradient Boosting also performed well, they did not achieve the same level of balance between correctly identifying failures and minimizing false predictions.

A key insight from this project is the importance of using appropriate evaluation metrics. Although all models achieved high accuracy, this metric was not sufficient due to the imbalance in the dataset. The F1-score proved to be a more meaningful measure, as it accounts for both false positives and false negatives. This highlights the need for careful metric selection when working with real-world datasets.

The confusion matrix analysis provided additional insight into model performance. While the model correctly classified the majority of non-failure cases and detected many failure cases, some failures were still missed. This limitation is particularly important in practical applications, where undetected failures can lead to serious consequences.

This project demonstrates that machine learning models, particularly ensemble methods, can be effectively applied to predictive maintenance problems. With proper data preprocessing and evaluation strategies, these models can provide valuable insights and support decision-making in real-world systems.

6. Future Work

Although the results of this project are strong, there are several areas where further improvements can be made.

One of the most important areas for improvement is handling class imbalance. While F1-score was used to evaluate model performance, additional techniques could be applied to improve the model's ability to detect failure cases. Methods such as SMOTE (Synthetic Minority Over-sampling Technique) or other resampling approaches could increase the representation of the minority class and improve recall.

Another potential improvement is feature engineering. While this project relied on the existing features in the dataset, creating new features based on domain knowledge could enhance model performance. For example, combining related sensor readings or identifying trends over time may provide additional predictive power.

Feature selection techniques could also be explored to reduce the dimensionality of the dataset. With 170 features, the dataset is highly complex, and some features may not contribute significantly to the prediction task. Reducing the number of features could improve computational efficiency and potentially enhance model performance.

Further, more advanced machine learning techniques could be investigated. Deep learning models, such as neural networks, may be capable of capturing more complex patterns in the data. However, these models require careful tuning and larger computational resources.

Hyperparameter tuning could also be expanded. While GridSearchCV was used in this project, other methods such as randomized search or Bayesian optimization could provide more efficient exploration of the parameter space.

F\uture work could involve testing the models on different datasets to evaluate their generalizability. Real-world deployment would require models that perform consistently across various conditions and data sources.

7. Extended Discussion

The results of this project highlight several important aspects of machine learning in real-world applications. One of the key challenges is dealing with high-dimensional data. As the number of features increases, the complexity of the model also increases, making it more difficult to identify meaningful patterns.

High-dimensional datasets often require models that can handle large amounts of information without overfitting. Ensemble methods such as Random Forest and XGBoost are particularly effective in this regard, as they combine multiple models to improve generalization.

Another important consideration is the handling of missing data. The APS dataset contains many missing values, which required careful preprocessing. The choice of median imputation proved to be effective, but other techniques such as K-nearest neighbors imputation or model-based imputation could be explored in future work.

Interpretability is another important factor. While complex models such as XGBoost provide high performance, they are often less interpretable than simpler models. In some applications, understanding the reasoning behind predictions is just as important as achieving high accuracy.

This project also demonstrates the importance of balancing model complexity and performance. While more complex models tend to achieve better results, they also require more computational resources and may be more difficult to deploy in real-world systems.

8. Practical Applications

The techniques and findings from this project have significant real-world applications, particularly in industries that rely on heavy machinery and transportation systems.

Predictive maintenance systems can be used to monitor equipment and detect potential failures before they occur. This allows organizations to perform maintenance at the optimal time, reducing costs and preventing unexpected breakdowns.

In the trucking industry, such systems could be used to monitor vehicle health and schedule maintenance proactively. This would improve operational efficiency, reduce downtime, and enhance safety.

The methods used in this project can also be applied to other domains. For example, similar techniques can be used in manufacturing to detect equipment faults, in energy systems to predict failures in power grids, or in healthcare to identify potential medical conditions.

The ability to analyze sensor data and predict outcomes has broad implications across many industries, making this an important area of research and development.

9. Limitations and Challenges

Despite the success of this project, several limitations should be acknowledged.

One major limitation is the imbalance in the dataset. While evaluation metrics such as F1-score help address this issue, the model still struggles to detect all failure cases. Improving recall remains a key challenge.

Another limitation is the lack of feature engineering. While the dataset contains many features, additional transformations or combinations of features could potentially improve performance.

The computational cost of training and tuning models is also a consideration. Ensemble methods, particularly XGBoost, require significant computational resources, which may limit their use in some applications.

The results are based on a single dataset. Evaluating the models on multiple datasets would provide a better understanding of their generalizability.

Real-world deployment introduces additional challenges, such as handling streaming data, adapting to changing conditions, and integrating with existing systems.

10. Final Remarks

This project demonstrates the effectiveness of machine learning models for predictive maintenance tasks. By carefully preprocessing data, selecting appropriate models, and using suitable evaluation metrics, it is possible to build systems that provide meaningful predictions.

The results highlight the importance of ensemble methods and the need to address class imbalance in real-world datasets. While there are still challenges to overcome, the findings of this project provide a strong foundation for further research and practical applications.

Machine learning continues to play an increasingly important role in industry, and its ability to analyze complex data and provide actionable insights makes it a valuable tool for improving efficiency and decision-making.

11. Additional Analysis of Model Behavior

Understanding how models behave under different conditions is essential for evaluating their effectiveness beyond standard performance metrics. In this project, different models exhibited varying strengths and weaknesses depending on how they handled the imbalanced dataset.

Linear models such as Logistic Regression struggled to capture complex relationships in the data. Although they provided a useful baseline, their performance was limited by their inability to model nonlinear interactions between features. This resulted in lower recall and F1-scores compared to more advanced models.

Tree-based models demonstrated improved performance by capturing nonlinear patterns. The Decision Tree model showed moderate success but was prone to overfitting due to its sensitivity to training data. Random Forest improved upon this by averaging multiple trees, reducing variance and improving stability.

Boosting methods, particularly Gradient Boosting and XGBoost, provided the best performance by iteratively correcting errors from previous models. XGBoost’s use of regularization and efficient optimization allowed it to outperform all other models.

This analysis highlights the importance of selecting models that align with the complexity of the dataset.

12. Trade-Off Between Precision and Recall

A critical aspect of classification problems, especially in imbalanced datasets, is the trade-off between precision and recall. These metrics often move in opposite directions, meaning that improving one may lead to a decrease in the other.

Precision reflects how accurate the model's positive predictions are. High precision means that when the model predicts a failure, it is likely correct. Recall, on the other hand, measures how many actual failures the model successfully detects.

In this project, XGBoost achieved a strong balance between precision and recall, resulting in the highest F1-score. However, the recall value indicates that some failures are still missed. This trade-off is important because missing a failure can have more serious consequences than incorrectly predicting one.

In practical applications, the choice between prioritizing precision or recall depends on the specific requirements of the system. For predictive maintenance, recall is often more important because detecting failures is critical. However, excessive false positives can also lead to unnecessary maintenance costs.

Balancing these metrics is therefore essential for building an effective model.

13. Computational Considerations

Additionally an important factor in machine learning projects is computational efficiency. Different models require varying amounts of computational resources for training and prediction.

Logistic Regression is relatively fast and computationally efficient, making it suitable for large datasets. However, its simplicity limits its performance in complex tasks.

Decision Trees are also relatively fast but can become inefficient as their depth increases. Random Forest and Gradient Boosting require more computational power due to the use of multiple trees.

XGBoost, while highly efficient compared to other boosting methods, still requires significant computational resources. Its performance benefits come at the cost of increased training time and complexity.

These considerations are important when selecting models for real-world applications, where computational constraints may impact feasibility.

14. Relevance to Machine Learning Principles

This project demonstrates several key principles of machine learning. One important principle is the bias-variance trade-off. Simple models such as Logistic Regression have high bias and may underfit the data, while complex models such as Decision Trees have low bias but high variance.

Ensemble methods such as Random Forest and XGBoost help address this trade-off by combining multiple models to reduce variance while maintaining low bias. This results in improved generalization and better performance on unseen data.

Another principle demonstrated in this project is the importance of data preprocessing. Proper handling of missing values, scaling, and data splitting significantly impacts model performance.

The project also highlights the role of hyperparameter tuning in optimizing model performance. Selecting appropriate parameter values can greatly improve accuracy and generalization.

15. Ethical and Practical Considerations

The use of machine learning in predictive maintenance raises important ethical and practical considerations. One concern is the reliability of predictions. Models must be accurate and robust to ensure that decisions based on their outputs are trustworthy.

Further consideration is the potential impact of incorrect predictions. False negatives, where failures are not detected, can lead to safety risks and financial losses. False positives, while less critical, can result in unnecessary maintenance and increased costs.

Transparency and interpretability are also important factors. In some applications, it may be necessary to explain how a model arrived at a particular prediction. Complex models such as XGBoost may require additional techniques to improve interpretability.

Data quality plays a crucial role in model performance. Inaccurate or incomplete data can lead to unreliable predictions, emphasizing the importance of proper data collection and preprocessing.
16. Lessons Learned

This project provided several important insights into the practical application of machine learning techniques. One of the most significant lessons learned is the importance of

understanding the dataset before applying models. The exploratory data analysis phase revealed key challenges such as class imbalance, missing values, and high dimensionality, all of which directly influenced model performance and design decisions.

Another important takeaway is the critical role of preprocessing. Proper handling of missing values and feature scaling significantly improved the effectiveness of the models. Without these steps, many algorithms would not have performed reliably. This highlights that data preparation is just as important as model selection in machine learning workflows.

The project also emphasized the importance of choosing appropriate evaluation metrics. Initially, accuracy appeared to indicate excellent performance across all models. However, further analysis revealed that accuracy was misleading due to the imbalanced dataset. The use of F1-score provided a more accurate representation of model performance, reinforcing the need to align evaluation metrics with the problem context.

Additionally, the comparison of multiple models demonstrated that more complex models do not always guarantee better results unless they are properly tuned. Hyperparameter tuning was essential for achieving optimal performance, particularly for ensemble methods such as Random Forest and XGBoost.

This project highlighted the importance of iterative experimentation. Machine learning is not a one-step process; it requires continuous refinement, testing, and evaluation. Each stage of the project contributed to a deeper understanding of both the data and the models, ultimately leading to improved results.

References

UCI Machine Learning Repository. (2016). APS Failure Dataset. https://archive.ics.uci.edu/dataset/414/ida2016challenge

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.